Python WebSocket -- Complete Notes (Beginner to Advanced)

1. What is WebSocket?

WebSocket is a communication protocol that provides full-duplex, bidirectional communication between a client and a server over a single persistent TCP connection. **বাংলায়:** WebSocket এমন একটি communication protocol যা মাধ্যমে Client এবং Server একই connection ব্যবহার করে যেকোনো সময় একে অপরের কাছে data পাঠাতে পারে।

Key Features

- Full-duplex communication.
- Bidirectional communication.
- Persistent connection.
- Low communication overhead.
- Real-time data transfer.
- Supports text and binary messages.
- Works over TCP.

WebSocket Protocols

``` text
ws://example.com/ws/
wss://example.com/ws/
```

- `ws://` = Unencrypted WebSocket.
- `wss://` = Secure WebSocket using TLS. Always use WSS in production.

---

2. HTTP vs WebSocket

Feature	HTTP	WebSocket
Communication	Request-response	Bidirectional
Connection	Usually request-based	Persistent
Data transfer	Client initiates	Both sides initiate
Server push	Requires additional techniques	Native support
Real-time	Polling/SSE	Yes
Protocol	HTTP	WS/WSS
Overhead	Repeated request overhead	Lower after handshake

HTTP Example

``` text
Client → Request → Server
Client ← Response ← Server
```

WebSocket Example

``` text
Client ↔ Server
Client ↔ Server
Client ↔ Server
```

Explanation: HTTP সাধারণত Client-এর request-এর পর Server response দেয়। WebSocket connection তৈরি হওয়ার পর উভয় পক্ষ স্বাধীনভাবে message পাঠাতে পারে।

---

3. How WebSocket Works

WebSocket communication works through the following steps: 1. Client sends an HTTP request. 2. Client requests protocol upgrade. 3. Server accepts the upgrade. 4. HTTP connection upgrades to WebSocket. 5. Persistent connection is established. 6. Client and Server exchange messages. 7. Either side can close the connection.

WebSocket Handshake

Client request:

``` http
GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: example-key
Sec-WebSocket-Version: 13
```

Server response:

``` http
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: ...
```

`101 Switching Protocols` means the server has accepted the protocol upgrade.

---

4. WebSocket Architecture

``` text
CLIENT
|
|
WebSocket Request
|
v
SERVER
|
Accept Connection
|
v
Persistent Connection
|
---------------------
| |
v v
Send Data Receive Data
| |
---------------------
|
v
Close Connection
```

Main Components

- Client: Initiates the connection.
- Server: Accepts the connection.
- Connection: Persistent communication channel.

- Message: Data exchanged between client and server.
- Frame: Unit of WebSocket protocol data transfer.

---

5. Python WebSocket Libraries

Library	Purpose
websockets	Async WebSocket server and client
FastAPI	WebSocket API
Django Channels	Django real-time communication
Flask-Sockk	Flask WebSocket integration
aiohttp	Async HTTP and WebSocket
Starlette	ASGI WebSocket support

Installation

``` bash
pip install websockets
```

FastAPI:

``` bash
pip install fastapi uvicorn
```

Django Channels:

``` bash
pip install channels channels-redis
```

---

6. Python Async Programming

WebSocket applications commonly use asynchronous programming.

What is Async Programming?

Asynchronous programming allows a program to perform other tasks while waiting for an operation to complete. **বাংলায়:** কোনো I/O operation শেষ

হওয়ার জন্য পুরো program-কে block না রেখে অন্য কাজ চালিয়ে যাওয়ার পদ্ধতি হলো asynchronous programming।

Important Keywords

Keyword	Meaning
async	Defines asynchronous function
await	Waits for asynchronous operation
asyncio	Python asynchronous programming library
create_task()	Schedules an asynchronous task
gather()	Runs multiple awaitables concurrently

Example

```python
import asyncio
async def say_hello():
    print("Hello")
async def main():
    await say_hello()
asyncio.run(main())
```

asyncio.sleep()

```python
import asyncio
async def main():
    print("Start")
    await asyncio.sleep(2)
    print("End")
asyncio.run(main())
```

Note: `asyncio.sleep()` does not block the entire event loop.

---

7. Basic WebSocket Server

File: server.py

```python
```

```
import asyncio
import websockets
async def handle_client(websocket):
    print("Client connected!")
    async for message in websocket:
        print(f"Received: {message}")
        await websocket.send(
            f"Server received: {message}"
        )
async def main():
    async with websockets.serve(
        handle_client,
        "localhost",
        8765
    ):
        print("WebSocket server running...")
        await asyncio.Future()
asyncio.run(main())
```

Explanation

- `asyncio`: Handles asynchronous operations.
- `websockets.serve()`: Creates a WebSocket server.
- `handle_client()`: Handles an individual client.
- `async for`: Receives incoming messages.
- `websocket.send()`: Sends messages.
- `asyncio.Future()`: Keeps the server running.

---

8. Basic WebSocket Client

File: client.py

``` python
import asyncio
import websockets
async def main():
    uri = "ws://localhost:8765"
    async with websockets.connect(uri) as websocket:
        await websocket.send("Hello Server!")
        response = await websocket.recv()
        print("Server:", response)
```

```
asyncio.run(main())
```

Run Application

Terminal 1:

``` bash
python server.py
```

Terminal 2:

``` bash
python client.py
```

Output:

``` text
Server: Server received: Hello Server!
```

---

9. Important WebSocket Methods

Method	Description
websockets.serve()	Creates WebSocket server
websockets.connect()	Connects WebSocket client
websocket.send()	Sends a message
websocket.recv()	Receives a message
websocket.close()	Closes connection
websocket.ping()	Sends ping
websocket.pong()	Sends pong
websocket.wait_closed()	Waits for connection closure

Example

``` python
await websocket.send("Hello")
message = await websocket.recv()
await websocket.close()
```

```

---

# 10. WebSocket with JSON

JSON is commonly used to exchange structured data.

### Server

``` python
import asyncio
import json
import websockets
async def handle_client(websocket):
    async for message in websocket:
        data = json.loads(message)
        username = data["username"]
        text = data["message"]
        response = {
            "username": username,
            "message": text,
            "status": "received"
        }
        await websocket.send(
            json.dumps(response)
        )
async def main():
    async with websockets.serve(
        handle_client,
        "localhost",
        8765
    ):
        await asyncio.Future()
asyncio.run(main())
```

JSON Message

``` json
{
"username": "Faruk",
"message": "Hello WebSocket"
}
```
```

```

### JSON Serialization

``` python
json.dumps(data)
```

Converts Python object to JSON string.

JSON Deserialization

``` python
json.loads(message)
```

Converts JSON string to Python object.

---

11. WebSocket with FastAPI

FastAPI provides built-in WebSocket support through Starlette.

Installation

``` bash
pip install fastapi uvicorn
```

Basic Implementation

File: main.py

``` python
from fastapi import FastAPI, WebSocket
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    while True:
        message = await websocket.receive_text()
        await websocket.send_text(
            f"Message received: {message}"
```
```

```
        )
```

Run Server

```bash
uvicorn main:app --reload
```

WebSocket URL:

```text
ws://127.0.0.1:8000/ws
```

Important FastAPI WebSocket Methods

```python
await websocket.accept()
await websocket.receive_text()
await websocket.receive_bytes()
await websocket.receive_json()
await websocket.send_text("Hello")
await websocket.send_bytes(b"Hello")
await websocket.send_json({
    "message": "Hello"
})
await websocket.close()
```

---

12. WebSocket Connection Lifecycle

A WebSocket connection has the following lifecycle: 1. CONNECTING 2. OPEN 3. CLOSING 4. CLOSED

Connection States

State	Description
CONNECTING	Connection is being established
OPEN	Connection is active
CLOSING	Connection is closing

CLOSED	Connection is closed

Connection Lifecycle Example

```python
await websocket.accept()
try:
    while True:
        message = await websocket.receive_text()
        await websocket.send_text(message)
except Exception:
    print("Connection error")
finally:
    await websocket.close()
```

In FastAPI, use `WebSocketDisconnect` for normal disconnect handling.

---

13. WebSocketDisconnect

When a client disconnects, FastAPI raises `WebSocketDisconnect`.

```python
from fastapi import FastAPI, WebSocket
from starlette.websockets import WebSocketDisconnect
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            message = await websocket.receive_text()
            await websocket.send_text(message)
    except WebSocketDisconnect:
        print("Client disconnected")
```

Why is it important? It helps clean up connection resources and
prevent stale connections.

---

14. Connection Manager

A Connection Manager manages multiple connected WebSocket clients.

Why Connection Manager?

- Track active clients.
- Accept connections.
- Remove disconnected clients.
- Broadcast messages.
- Manage client connections.

Implementation

``` python
from fastapi import FastAPI, WebSocket
from starlette.websockets import WebSocketDisconnect
app = FastAPI()
class ConnectionManager:
    def __init__(self):
        self.active_connections = []
    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)
    def disconnect(self, websocket: WebSocket):
        if websocket in self.active_connections:
            self.active_connections.remove(websocket)
    async def send_personal_message(
        self,
        message: str,
        websocket: WebSocket
    ):
        await websocket.send_text(message)
    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)
manager = ConnectionManager()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await manager.connect(websocket)
    try:
        while True:
            message = await websocket.receive_text()
            await manager.broadcast(message)
```

```
    except WebSocketDisconnect:
        manager.disconnect(websocket)
```

Important: This in-memory manager works for a single application process. Multiple workers need shared coordination, typically Redis or another messaging system.

---

15. Broadcasting Messages

Broadcasting means sending one message to all connected clients.

Example

``` python
async def broadcast(self, message: str):
    for connection in self.active_connections:
        await connection.send_text(message)
```

Use Cases

- Group chat.
- Live notifications.
- Multiplayer games.
- Real-time dashboards.
- Live sports updates.

Personal Message vs Broadcast

Personal Message	Broadcast
One client	Multiple clients
Private communication	Group communication
send_personal_message()	broadcast()

---

16. WebSocket Authentication

Authentication ensures that only authorized users can connect.

Common Authentication Methods

- JWT Authentication.
- Session Authentication.
- OAuth2.
- API Token.

JWT Authentication Concept

1. User logs in.
2. Server generates JWT.
3. Client receives JWT.
4. Client connects to WebSocket.
5. Server validates token.
6. Connection is accepted or rejected.

FastAPI Example

```` python
from fastapi import FastAPI, WebSocket, Query
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(
    websocket: WebSocket,
    token: str = Query(...)
):
    if token!= "secret-token":
        await websocket.close(code=1008)
        return
        await websocket.accept()
        while True:
            message = await websocket.receive_text()
            await websocket.send_text(message)
````

This is only a demonstration. Real applications must validate JWT signatures, expiration, issuer, audience and permissions.

---

17. WebSocket Authorization

Authentication and authorization are different. **Authentication:** Who are you? **Authorization:** What are you allowed to do? Example: - User

A can access Room 1. - User B can access Room 2. - Admin can access both rooms. Always validate permissions before allowing access to a room or performing an action.

---

18. WebSocket Ping and Pong

Ping and Pong help detect whether a connection is still alive.

Ping

A message used to check connection availability.

Pong

A response to a Ping frame.

Why Heartbeat?

- Detect dead connections.
- Maintain connection health.
- Detect network failures.
- Improve reliability. The Python `websockets` library supports automatic ping and pong configuration.

``` python
async with websockets.serve(
    handle_client,
    "localhost",
    8765,
    ping_interval=20,
    ping_timeout=20
):
    await asyncio.Future()
```

- `ping_interval`: Time between automatic pings.
- `ping_timeout`: Maximum waiting time for pong.

---

19. WebSocket Reconnection

Sometimes connections fail because of: - Internet disconnection. - Server restart. - Network timeout. - Proxy interruption. Clients should reconnect when appropriate.

Simple Reconnection Example

``` python
import asyncio
import websockets
async def connect():
    uri = "ws://localhost:8765"
    while True:
        try:
            async with websockets.connect(uri) as websocket:
                print("Connected!")
                while True:
                    message = await websocket.recv()
                    print(message)
        except Exception as error:
            print("Connection failed:", error)
            await asyncio.sleep(5)
asyncio.run(connect())
```

Production applications should use exponential backoff, maximum retry limits, jitter and authentication refresh where necessary.

---

20. Redis Pub/Sub with WebSocket

Redis Pub/Sub allows multiple application instances to exchange messages.

Why Redis?

Consider an application running multiple server instances:

``` text
CLIENTS
|
LOAD BALANCER
/ \
/ \
```

SERVER 1 SERVER 2
| |
+----REDIS-+
```

Without shared messaging, a message received by Server 1 may not reach clients connected to Server 2. Redis Pub/Sub solves this coordination problem.

### Installation

``` bash
pip install redis
```

Redis Connection

``` python
import redis.asyncio as redis
client = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)
```

Publish

``` python
await client.publish(
    "chat",
    "Hello Everyone"
)
```

Subscribe

``` python
pubsub = client.pubsub()
await pubsub.subscribe("chat")
async for message in pubsub.listen():
    if message["type"] == "message":
        print(message["data"])
```
```

Important: Redis Pub/Sub is not durable. Disconnected subscribers can miss messages. For persistent event delivery, consider Redis Streams or another message broker.

---

21. WebSocket Rooms

A room allows a group of users to communicate with each other.

Example

```text
Room 1:
User A
User B
User C
Room 2:
User D
User E
```

Room Manager Concept

```python
class RoomManager:
    def __init__(self):
        self.rooms = {}
    async def join_room(self, room_id, websocket):
        await websocket.accept()
        if room_id not in self.rooms:
            self.rooms[room_id] = []
            self.rooms[room_id].append(websocket)
    def leave_room(self, room_id, websocket):
        if room_id in self.rooms:
            if websocket in self.rooms[room_id]:
                self.rooms[room_id].remove(websocket)
    async def send_to_room(self, room_id, message):
        for connection in self.rooms.get(room_id, []):
            await connection.send_text(message)
```

This is a basic example. Production room managers also need disconnect

cleanup, authorization and concurrent-send handling.

---

22. WebSocket Message Types

WebSocket supports different types of messages.

Text Message

```python
await websocket.send_text("Hello")
```

Binary Message

```python
await websocket.send_bytes(b"Hello")
```

JSON Message

```python
await websocket.send_json({
    "username": "Faruk",
    "message": "Hello"
})
```

Receive Messages

```python
text = await websocket.receive_text()
binary = await websocket.receive_bytes()
data = await websocket.receive_json()
```

---

23. WebSocket Close Codes

Code	Meaning
1000	Normal closure

1001 Going away
1002 Protocol error
1003 Unsupported data
1006 Abnormal closure (reserved)
1007 Invalid frame payload data
1008 Policy violation
1009 Message too big
1011 Internal server error

Example

```python
await websocket.close(code=1000)
```

---

24. WebSocket Security Best Practices

Security is important in production applications.

Best Practices

1. Always use WSS.
2. Validate authentication tokens.
3. Validate Origin headers.
4. Validate user permissions.
5. Implement rate limiting.
6. Set message size limits.
7. Validate incoming JSON.
8. Handle exceptions properly.
9. Prevent unauthorized room access.
10. Avoid logging sensitive data.
11. Implement connection limits.
12. Protect against denial-of-service attacks.

Message Validation Example

```python
from pydantic import BaseModel, ValidationError
class ChatMessage(BaseModel):
    username: str
    message: str
    def validate_message(data):
```

```
        try:
            return ChatMessage.model_validate(data)
        except ValidationError:
            return None
```

---

25. WebSocket vs Server-Sent Events (SSE)

Feature	WebSocket	SSE
Communication	Bidirectional	Server to client
Protocol	WebSocket	HTTP
Real-time	Yes	Yes
Client messages	Yes	Separate HTTP request
Reconnection	Application-managed	Built-in browser support
Use case	Chat, gaming	Live feeds, notifications

---

26. WebSocket vs Django Channels

Feature	WebSocket	Django Channels
Type	Protocol	Django framework extension
Real-time communication	Yes	Yes
Django integration	Manual	Integrated
Group communication	Custom	Channel groups
Redis integration	Custom	Channel layer
ASGI	Compatible	Built around ASGI

Django Channels Installation

``` bash
pip install channels channels-redis
```

Add to INSTALLED_APPS

``` python
INSTALLED_APPS = [
    "channels",
    "django.contrib.admin",
```

```
    "django.contrib.auth",
    "django.contrib.contenttypes",
]
```

ASGI Configuration

``` python
ASGI_APPLICATION = "config.asgi.application"
```

---

27. WebSocket with Django Channels

Consumer

A Consumer handles WebSocket connections and events in Django Channels.
File: consumers.py

``` python
from channels.generic.websocket import AsyncWebsocketConsumer
import json
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.accept()
    async def disconnect(self, close_code):
        print("Disconnected")
    async def receive(self, text_data):
        data = json.loads(text_data)
        message = data["message"]
        await self.send(
            text_data=json.dumps({
                "message": message
            })
        )
```

Routing

File: routing.py

``` python
from django.urls import path
```

```
from .consumers import ChatConsumer
websocket_urlpatterns = [
    path("ws/chat/", ChatConsumer.as_asgi()),
]
```

ASGI Configuration

``` python
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from chat.routing import websocket_urlpatterns
os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)
django_asgi_app = get_asgi_application()
application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AuthMiddlewareStack(
        URLRouter(websocket_urlpatterns)
    ),
})
```

---

28. Django Channels Groups

Groups allow multiple WebSocket clients to receive the same message.

Group Add

``` python
await self.channel_layer.group_add(
    "chat_room",
    self.channel_name
)
```

Group Send

``` python
await self.channel_layer.group_send(
    "chat_room",
    {
        "type": "chat_message",
        "message": "Hello Everyone"
    }
)
```

Group Discard

``` python
await self.channel_layer.group_discard(
    "chat_room",
    self.channel_name
)
```

Event Handler

``` python
async def chat_message(self, event):
    await self.send(
        text_data=json.dumps({
            "message": event["message"]
        })
    )
```

---

29. WebSocket with Redis Channel Layer

Install:

``` bash
pip install channels-redis
```

Django settings.py:

``` python
CHANNEL_LAYERS = {
```

```
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [
                ("127.0.0.1", 6379)
            ],
        },
    },
}
```

Redis provides shared communication between application workers.

---

30. WebSocket Error Handling

Common problems: \| Error \| Possible Cause \| \|---\|---\| \|
Connection refused \| Server not running \| \| Connection closed \|
Server/client disconnected \| \| Invalid handshake \| Incorrect
handshake \| \| Timeout \| Network or server delay \| \| Authentication
failed \| Invalid token \| \| 403 Forbidden \| Origin or permission
rejected \| \| 404 Not Found \| Incorrect endpoint \|

Exception Handling

``` python
from fastapi import WebSocket
from starlette.websockets import WebSocketDisconnect
@app.websocket("/ws")
async def endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            message = await websocket.receive_text()
            await websocket.send_text(message)
    except WebSocketDisconnect:
        print("Client disconnected")
    except Exception as error:
        print("Error:", error)
```

---

31. WebSocket Performance Optimization

1. Use asynchronous programming.
2. Avoid blocking operations.
3. Use Redis for distributed messaging.
4. Limit message sizes.
5. Use efficient JSON serialization.
6. Implement connection limits.
7. Use appropriate heartbeat settings.
8. Avoid unnecessary broadcasting.
9. Monitor active connections.
10. Use load balancing.
11. Handle backpressure.
12. Optimize database queries.

---

32. WebSocket Deployment

For production deployment, consider: - ASGI server. - Nginx reverse proxy. - HTTPS/TLS. - Redis. - Load balancer. - Authentication. - Monitoring. - Logging. - Connection limits.

Example Uvicorn Command

``` bash
uvicorn main:app --host 0.0.0.0 --port 8000
```

Nginx Configuration Concept

``` nginx
location /ws/ {
proxy_pass http://127.0.0.1:8000;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_set_header Host $host;
proxy_read_timeout 3600s;
}
```

---

33. Real-World WebSocket Use Cases

1. Real-time Chat Application.
2. Live Notification System.
3. Live Stock Market Dashboard.
4. Online Multiplayer Games.
5. Real-time Location Tracking.
6. Live Delivery Tracking.
7. Collaborative Document Editing.
8. Live Sports Score Updates.
9. Online Customer Support.
10. Real-time Monitoring Dashboard.

---

34. Beginner to Advanced Practice Projects

Beginner Level

- Echo WebSocket Server.
- Simple WebSocket Client.
- Real-time Message Exchange.
- JSON Message Server.
- Basic Notification System.

Intermediate Level

- Real-time Chat Application.
- Multiple Client Broadcasting.
- User Authentication.
- Online User Status.
- Chat Rooms.
- Private Messaging.

Advanced Level

- FastAPI + Redis Chat Application.
- Django Channels Chat Application.
- Group Messaging.
- JWT Authentication.
- Redis Pub/Sub.
- Real-time Notification Backend.
- Multi-worker WebSocket Architecture.

Expert Level

- Distributed WebSocket Architecture.
- Horizontal Scaling.
- Load Balancing.
- Connection Recovery.
- Message Queue Integration.
- Production Monitoring.
- High-Concurrency WebSocket Server.

---

35. Interview Questions and Answers

Q1. What is WebSocket?

WebSocket is a full-duplex communication protocol that provides persistent, bidirectional communication between a client and a server.

Q2. What is the difference between HTTP and WebSocket?

HTTP follows a request-response model, whereas WebSocket supports bidirectional communication through a persistent connection.

Q3. What is a WebSocket handshake?

The initial HTTP Upgrade process that establishes a WebSocket connection.

Q4. What does 101 Switching Protocols mean?

It indicates that the server has accepted the protocol upgrade request.

Q5. What is full-duplex communication?

Both communication parties can send and receive data simultaneously.

Q6. Why use async and await?

They enable asynchronous execution, helping applications handle I/O operations efficiently.

Q7. What is broadcasting?

Sending a single message to multiple connected clients.

Q8. What is a Connection Manager?

A component that manages active WebSocket connections.

Q9. What is WebSocket authentication?

The process of verifying the identity of a client before allowing access.

Q10. What is WebSocket authorization?

The process of checking whether an authenticated client has permission to perform an action.

Q11. What is Redis Pub/Sub?

A messaging mechanism where publishers send messages to channels and subscribers receive messages from those channels.

Q12. What is Django Channels?

A Django extension that supports asynchronous protocols, including WebSockets.

Q13. What is a WebSocket Consumer?

A Django Channels component that handles connection, disconnection and incoming events.

Q14. What is a WebSocket heartbeat?

A periodic ping/pong mechanism used to detect inactive connections.

Q15. What is WSS?

Secure WebSocket communication over TLS.

Q16. What happens when a client disconnects?

The server should detect the disconnect and clean up the client's connection state.

Q17. What are WebSocket rooms?

Logical groups of connected users who can exchange messages with each other.

Q18. How do you scale WebSocket applications?

Use shared messaging infrastructure such as Redis, multiple ASGI workers, load balancing and proper connection management.

Q19. What is the difference between WebSocket and SSE?

WebSocket supports bidirectional communication, while SSE provides server-to-client streaming over HTTP.

Q20. Is WebSocket suitable for real-time chat applications?

Yes. WebSocket is well suited to real-time chat because both clients and servers can send messages over persistent connections.

---

36. Important Terminology

Term	Meaning
WebSocket	Bidirectional communication protocol
Full-duplex	Simultaneous two-way communication
Handshake	Initial connection negotiation
Upgrade	Switch from HTTP to WebSocket
Persistent Connection	Connection remains open
Frame	Protocol-level data unit
Message	Data exchanged
Ping	Connection health check
Pong	Response to Ping
Broadcasting	Send to multiple clients
Consumer	Django Channels event handler
Channel Layer	Communication between consumers
Redis Pub/Sub	Publish-subscribe messaging
ASGI	Asynchronous Server Gateway Interface
WSS	Secure WebSocket
Reconnection	Re-establish a closed connection
Heartbeat	Periodic connection health check

---

37. WebSocket Learning Roadmap

Level	Topics
01	HTTP vs WebSocket
02	WebSocket Architecture
03	Handshake
04	Python Async Programming
05	WebSocket Server
06	WebSocket Client
07	JSON Communication
08	FastAPI WebSocket
09	Connection Manager
10	Broadcasting
11	Authentication
12	Authorization
13	Ping/Pong
14	Reconnection
15	WebSocket Rooms
16	Redis Pub/Sub
17	Django Channels
18	Channel Groups
19	Security
20	Production Deployment

---

38. Official Documentation

1. Python WebSockets: https://websockets.readthedocs.io/
2. FastAPI WebSockets: https://fastapi.tiangolo.com/advanced/websockets/
3. Django Channels: https://channels.readthedocs.io/
4. Redis Pub/Sub: https://redis.io/docs/latest/develop/pubsub/
5. Python Asyncio: https://docs.python.org/3/library/asyncio.html
6. ASGI: https://asgi.readthedocs.io/

---

39. Final Summary

WebSocket is an important technology for building real-time

applications.

Most Important Concepts

- HTTP vs WebSocket.
- Handshake.
- Full-duplex communication.
- Async and Await.
- WebSocket Server and Client.
- FastAPI WebSocket.
- Connection Manager.
- Broadcasting.
- Authentication and Authorization.
- Ping and Pong.
- Redis Pub/Sub.
- Django Channels.
- WebSocket Security.
- Production Deployment.

Recommended Learning Sequence

Python Async Programming → WebSocket Fundamentals → Python WebSocket → FastAPI WebSocket → Connection Manager → Broadcasting → Authentication → Redis Pub/Sub → Django Channels → Production Deployment. **Practice Goal:** Build a complete Real-Time Chat Application using Python, FastAPI, WebSocket, Redis and PostgreSQL.

Additional Django Channels and Advanced WebSocket Notes

> Note: This section expands topics already introduced earlier. Some
> Bangla text in the supplied source had broken character spacing; that
> text is preserved where it could not be reconstructed reliably.

40. WebSocket with Django Channels

Django-তে real-time WebSocket application তৈরি করতে Django Channels ব্যবহার করা হয়।

What is Django Channels?

Django Channels হলো Django-এর asynchronous communication support বাড়ানোর একটি extension। এটি WebSocket, background communication এবং অন্যান্য

asynchronous protocol পরিচালনা করতে সাহায্য করে।

Installation

``` bash
pip install channels
```

Redis channel layer ব্যবহার করতে:

``` bash
pip install channels-redis
```

Project Structure

``` text
myproject/
│
├── manage.py
│
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── asgi.py
│   └── urls.py
│
└── chat/
    ├── __init__.py
    ├── consumers.py
    └── routing.py
```

Step 1: settings.py

``` python
INSTALLED_APPS = [
    "channels",
    "chat",
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
```

```
    "django.contrib.staticfiles",
]
ASGI_APPLICATION = "config.asgi.application"
```

Step 2: asgi.py

``` python
import os
from django.core.asgi import get_asgi_application
from channels.routing import (
    ProtocolTypeRouter,
    URLRouter
)
from channels.auth import AuthMiddlewareStack
from chat.routing import websocket_urlpatterns
os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)
django_asgi_app = get_asgi_application()
application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AuthMiddlewareStack(
        URLRouter(
            websocket_urlpatterns
        )
    ),
})
```

Step 3: consumers.py

Consumer হলো Django Channels-এর এমন একটি component, যা connection এবং message পরিচালনা করে।

``` python
import json
from channels.generic.websocket import (
    AsyncWebsocketConsumer
)
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.accept()
```

```
        await self.send(
            text_data=json.dumps({
                "message": "Connected successfully"
            })
        )
    async def disconnect(self, close_code):
        print("Disconnected:", close_code)
    async def receive(self, text_data):
        data = json.loads(text_data)
        message = data.get("message", "")
        await self.send(
            text_data=json.dumps({
                "message": message
            })
        )
```

Step 4: routing.py

``` python
from django.urls import path
from .consumers import ChatConsumer
websocket_urlpatterns = [
    path(
        "ws/chat/",
        ChatConsumer.as_asgi()
    ),
]
```

Step 5: Run Server

Development-এর জন্য Daphne install করতে পারো:

``` bash
pip install daphne
```

``` bash
daphne -p 8000 config.asgi:application
```

Important Consumer Methods

Method	Description
`connect()`	Client connect হলে execute হয়
`disconnect()`	Client disconnect হলে execute হয়
`receive()`	Message receive হলে execute হয়
`send()`	Message পাঠায়
`accept()`	Connection গ্রহণ করে
`close()`	Connection বন্ধ করে

Sync vs Async Consumer

Sync Consumer

```python
from channels.generic.websocket import (
    WebsocketConsumer
)
class MyConsumer(WebsocketConsumer):
    def connect(self):
        self.accept()
    def receive(self, text_data):
        self.send(text_data=text_data)
    def disconnect(self, close_code):
        pass
```

Async Consumer

```python
from channels.generic.websocket import (
    AsyncWebsocketConsumer
)
class MyConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.accept()
    async def receive(self, text_data):
        await self.send(text_data=text_data)
    async def disconnect(self, close_code):
        pass
```

Difference

Sync Consumer	Async Consumer
Synchronous	Asynchronous
Blocking operations হতে পারে	Non-blocking I/O support করে
Normal Django sync code-এর জন্য	Async operations-এর জন্য সুবিধাজনক
সুবিধাজনক	

---

41. WebSocket Groups and Broadcasting

What is Broadcasting?

Broadcasting হলো একটি message একাধিক connected client-এর কাছে পাঠানো।
Example: একটি chat room-এ ১০ জন user connected আছে। একজন user message পাঠালে বাকি user-রা সেই message পাবে।

Django Channels Group

একটি group-এর মধ্যে একাধিক WebSocket connection রাখা যায়।

Consumer Example

```python
import json
from channels.generic.websocket import (
    AsyncWebsocketConsumer
)
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.room_group_name = "chat_room"
        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name
        )
        await self.accept()
    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(
            self.room_group_name,
            self.channel_name
        )
```

```
    async def receive(self, text_data):
        data = json.loads(text_data)
        message = data["message"]
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                "type": "chat_message",
                "message": message
            }
        )
    async def chat_message(self, event):
        await self.send(
            text_data=json.dumps({
                "message": event["message"]
            })
        )
```

Important Methods

group_add()

Client-কে group-এ যুক্ত করে।

``` python
await self.channel_layer.group_add(
    group_name,
    channel_name
)
```

group_discard()

Client-কে group থেকে সরিয়ে দেয়।

``` python
await self.channel_layer.group_discard(
    group_name,
    channel_name
)
```

group_send()

Group-এর সব member-এর কাছে event পাঠায়।

```python
await self.channel_layer.group_send(
    group_name,
    {
        "type": "chat_message",
        "message": "Hello"
    }
)
```

Channel Layer Event Type

```python
{
    "type": "chat_message"
}
```

এখা নে `type` হলো handler method-এর নাম নির্ধারণ করে। `chat_message` type-এর জন্য `chat_message()` method execute হবে।

---

42. Redis with WebSocket

What is Redis?

Redis হলো একটি in-memory data store, যা caching, messaging এবং অন্যান্য দ্রুত data operations-এর জন্য ব্যবহার করা হয়। Django Channels-এ Redis channel layer হিসেবে ব্যবহার করা যায়।

Why Redis?

ধরো, তোমার application-এ দুটি WebSocket server রয়েছে।

```text
CLIENTS
|
------------
| |
Server 1 Server 2
| |
```

```
------------
|
Redis
```

একটি Client Server 1-এর সঙ্গে connected। অন্য Client Server 2-এর সঙ্গে connected। Server 1 থেকে পাঠানো message Server 2-এর Client-দের কাছে পৌঁছাতে Redis channel layer সাহায্য করতে পারে।

Install Redis Channel Layer

``` bash
pip install channels-redis
```

settings.py

``` python
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": (
            "channels_redis.core.RedisChannelLayer"
        ),
        "CONFIG": {
            "hosts": [
                "redis://127.0.0.1:6379/0"
            ],
        },
    },
}
```

Redis Responsibilities

- Channel layer messaging
- Cross-server communication
- Group broadcasting
- Temporary message distribution

Important Note

Redis channel layer permanent message storage-এর বিকল্প নয়। Chat history, user information বা order information স্থায়ী ভাবে রাখতে PostgreSQL বা অন্য database ব্যবহার করতে হবে।

---

43. WebSocket Authentication

WebSocket connection-এর ক্ষেত্রে ও authentication প্রয়োজন। কারণ যেকোনো user যেন অন্য user-এর private message বা protected data access করতে না পারে।

Authentication Methods

1. Session Authentication
2. Token Authentication
3. JWT Authentication
4. Custom Authentication Middleware

Django Channels Session Authentication

`AuthMiddlewareStack` ব্যবহার করলে WebSocket scope-এ session-based authenticated user পাওয়া যায়। Example:

```python
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        user = self.scope["user"]
        if user.is_authenticated:
            await self.accept()
        else:
            await self.close()
```

JWT Authentication

JWT ব্যবহার করলে token validate করতে হবে। Concept:

```text
Client
|
| JWT Token
|
WebSocket Server
|
| Validate Token
|
Authenticated User
```

```

**Important:** Browser-এর native WebSocket constructor arbitrary custom headers সেট করার সুযোগ দেয় না। তাই browser client-এর ক্ষেত্রে secure session cookie, অথবা carefully designed token authentication flow ব্যবহার করা যায়। URL query string-এ token পাঠালে server, proxy বা access log-এ token থেকে যে তে পারে। তাই এটি এড়িয়ে চলা বা অত্য ন্ত সতর্কভাবে পরিচালনা করা উচিত।

---

# 44. WebSocket Security

WebSocket application-এ security খুব গুরুত্বপূর্ণ।

## 1. Use WSS

Production-এ encrypted WebSocket ব্যবহার করো।

``` text
wss://example.com/ws/
```

2. Authentication

প্রতিটি connection-এর user identity যাচাই করো।

3. Authorization

User যে room বা resource access করার অনুমতি রাখে শুধু সেটিই access করতে দাও।

4. Origin Validation

অপরিচিত website থেকে WebSocket connection গ্রহণ করা বন্ধ করো। Django Channels-এ:

``` python
from channels.security.websocket import (
    AllowedHostsOriginValidator
)
application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
```
```

```
        AuthMiddlewareStack(
            URLRouter(websocket_urlpatterns)
        )
    ),
})
```

5. Input Validation

Client থেকে আসা message validate করো।

``` python
message = data.get("message")
if not isinstance(message, str):
    return
    if len(message) > 1000:
        return
```

6. Rate Limiting

একজন user যেন অতিরিক্ত message পাঠিয়ে server overload করতে না পারে।

7. Message Size Limit

অতিরিক্ত বড় message reject করো।

8. Avoid Sensitive Logging

Password, JWT token বা private message log করা উচিত নয়।

---

45. Polling vs Long Polling vs SSE vs WebSocket

1. Polling

Client নির্দিষ্ট সময় পরপর Server-কে request পাঠায়।

``` text
Client ---> Server
Client ---> Server
Client ---> Server
```

Example:

```javascript
setInterval(async () => {
const response = await fetch("/api/updates/");
const data = await response.json();
console.log(data);
}, 5000);
```

2. Long Polling

Client request পাঠায়। Server data পাওয়া পর্যন্ত বা timeout হওয়া পর্যন্ত request ধরে রাখে। Response পাওয়ার পর Client আবার request পাঠায়।

3. SSE (Server-Sent Events)

Server থেকে Client-এর দিকে একমুখী real-time event stream পাঠানো হয়।
Client সাধারণ HTTP connection ব্যবহার করে event গ্রহণ করে।

4. WebSocket

Client এবং Server উভয় দি ক থেকে message পাঠানো যায়।

Comparison

Feature	Polling	Long Polling	SSE	WebSocket
Direction	Client request	Server response	Server to Client	Both directions
Persistent	No	Request-based	Yes	Yes
Real-time	Limited	Better	Yes	Yes
Browser support	Yes	Yes	Yes	Yes
Best for	Simple updates	Older real-time systems	Notifications	Chat and gaming

---

46. Ping and Pong

Ping এবং Pong WebSocket-এর control frame। এগুলো connection সচল আছে কি না পরীক্ষা করতে সাহায্য করে।

Ping

একটি পক্ষ Ping পাঠায়।

Pong

অন্য পক্ষ Pong দি য়ে response দেয়।

``` text
Client Server
|------ Ping --------->|
| |
|<----- Pong ----------|
| |
```

Why Needed?

কখনো network disconnect হয়ে যায়, কিন্তু application বুঝতে পারে না।
Heartbeat mechanism ব্যবহার করলে dead connection শনাক্ত করা সহজ হয়।
Note: Browser-এর JavaScript WebSocket API সরাসরি Ping/Pong control frame তৈরি বা পড়া র interface দেয় না। সাধারণত browser implementation এগুলো protocol level-এ handle করে।

---

47. Error Handling and Reconnection

Network problem হলে WebSocket connection disconnect হতে পারে। তাই reconnect logic থাকা গুরুত্বপূর্ণ।

Basic Reconnection Example

``` javascript
let socket;
function connectWebSocket() {
socket = new WebSocket(
```

```
"ws://localhost:8000/ws/"
);
socket.onopen = () => {
console.log("Connected");
};
socket.onmessage = (event) => {
console.log(event.data);
};
socket.onclose = () => {
console.log("Disconnected");
setTimeout(
connectWebSocket,
3000
);
};
socket.onerror = (error) => {
console.log("Error:", error);
};
}
connectWebSocket();
```

Exponential Backoff

প্রতি বার reconnect fail হলে অপেক্ষা র সময় বাড়ানো হয়। Example:

``` text
Attempt 1: 1 second
Attempt 2: 2 seconds
Attempt 3: 4 seconds
Attempt 4: 8 seconds
Attempt 5: 16 seconds
```

Improved Reconnection Example

``` javascript
let socket;
let retryDelay = 1000;
const MAX_DELAY = 30000;
function connect() {
socket = new WebSocket(
"wss://example.com/ws/"
);
```

```
socket.onopen = () => {
console.log("Connected");
retryDelay = 1000;
};
socket.onmessage = (event) => {
console.log(event.data);
};
socket.onclose = () => {
const delay =
retryDelay + Math.random() * 1000;
setTimeout(connect, delay);
retryDelay = Math.min(
retryDelay * 2,
MAX_DELAY
);
};
}
connect();
```

Production application-এ intentional close, authentication failure এবং temporary network failure আলাদাভাবে handle করা উচিত।

---

48. WebSocket Performance Optimization

অনেক concurrent connection থা কলে performance গুরুত্বপূর্ণ হয়ে ওঠে।

1. Use Async Programming

I/O-heavy application-এ async programming ব্যবহার করো।

2. Reduce Message Size

অপ্রয়োজনীয় বড় JSON message পাঠিও না।

3. Avoid Unnecessary Broadcasting

শুধু প্রয়োজনী য় user বা group-কে message পা ঠা ও।

4. Use Redis

Multiple server-এর মধ্যে message distribute করতে Redis channel layer

ব্যবহার করতে পারো।

5. Limit Connections

প্রতি user বা IP-এর connection সংখ্যা সীমিত রাখো।

6. Handle Disconnects

Disconnected client-এর resources release করো।

7. Use Pagination

পুরোনো chat history একবারে না পাঠিয়ে pagination ব্যবহার করো।

8. Monitor Connections

Active connection, message throughput এবং error rate monitor করো।

---

49. WebSocket Scaling

ধরো, তোমার application-এ ১০,০০০ concurrent user রয়েছে। একটি server দি য়ে সব connection পরিচালনা করা কঠিন হতে পারে। তখন multiple server ব্যবহার করা যায়।

Architecture

``` text
USERS
|
LOAD BALANCER
|
------------------
| | |
Server 1 Server 2 Server 3
| | |
------------------
|
REDIS
|
DATABASE
```

Important Components

Load Balancer

Incoming connection বিভিন্ন server-এ distribute করে।

Multiple Application Servers

একাধিক server connection handle করে।

Redis

Server-এর মধ্যে event বা message exchange করতে সাহায্য করে।

Database

Permanent information সংরক্ষণ করে।

Sticky Sessions

কিছু deployment setup-এ একই client-এর connection নির্দিষ্ট server-এর সঙ্গে ধরে রাখা র প্রয়োজন হতে পারে। তবে WebSocket handshake সম্পন্ন হওয়ার পর connection সাধারণত সেই server-এর সঙ্গেই থাকে।

---

50. WebSocket Use Cases

1. Chat Application

Real-time messaging।

2. Live Notification

নতুন notification সঙ্গে সঙ্গে পাঠানো।

3. Online Gaming

Real-time player movement এবং game state update।

4. Live Stock Market

Stock price update।

5. Live Tracking

Delivery বা vehicle tracking।

6. Collaborative Editor

একাধিক user একই document edit করতে পারে।

7. Real-time Dashboard

Live analytics এবং monitoring।

8. Customer Support

Live customer-agent communication।

---

51. WebSocket Best Practices

1. Production-এ WSS ব্যবহার করো।
2. Authentication implement করো।
3. Authorization check করো।
4. Origin validate করো।
5. Incoming message validate করো।
6. Message size limit রাখো।
7. Rate limiting implement করো।
8. Proper error handling রাখো।
9. Reconnection strategy তৈরি করো।
10. Dead connection detect করো।
11. Unnecessary broadcasting এড়িয়ে চলো।
12. Sensitive data log করো না।
13. Database operation carefully handle করো।
14. Connection cleanup নিশ্চিত করো।
15. Monitoring এবং logging implement করো।

---

52. Common WebSocket Errors

1. Connection Refused

কারণ: - Server চালু নেই। - ভুল port ব্যবহার করা হয়েছে। - Endpoint ভুল।
সমাধান: - Server status check করো। - URL এবং port check করো।

2. WebSocket is Closed Before the Connection is Established

কারণ: - Server handshake reject করেছে। - Server crash করেছে। - Authentication failure হয়েছে। সমাধান: - Server logs check করো। - Authentication এবং routing check করো।

3. 403 Forbidden

কারণ: - Origin validation failure। - Authentication বা permission failure। সমাধান: - Allowed hosts check করো। - User authentication এবং authorization check করো।

4. 404 Not Found

কারণ: - WebSocket routing path ভুল। সমাধান: - URL pattern check করো।

5. 500 Internal Server Error

কারণ: - Server-side exception। সমাধান: - Application logs check করো। - Consumer বা endpoint code debug করো।

6. WebSocketDisconnect

কারণ: - Client connection বন্ধ করেছে। - Network connection lost হয়েছে। সমাধান: - Disconnect event handle করো।

---

53. WebSocket Interview Questions

Beginner Level

Q1. What is WebSocket?

Answer: WebSocket is a communication protocol that provides full-duplex communication between a client and server over a persistent connection.
বাংলায়: Client এবং Server-এর মধ্যে real-time bidirectional communication তৈরি করে।

Q2. What is the difference between HTTP and WebSocket?

Answer: HTTP follows a request-response model, while WebSocket provides full-duplex communication over a persistent connection.

Q3. What is ws://?

Answer: It is the WebSocket URI scheme for an unencrypted WebSocket connection.

Q4. What is wss://?

Answer: It is the secure WebSocket URI scheme that uses TLS encryption.

Q5. What is a WebSocket handshake?

Answer: It is the initial process used to establish a WebSocket connection.

Q6. What does HTTP 101 mean?

Answer: 101 Switching Protocols indicates that the server accepted the protocol upgrade request.

Q7. What is full-duplex communication?

Answer: Both client and server can send and receive data independently at the same time.

Intermediate Level

Q8. What is Broadcasting?

Answer: Broadcasting means sending one message to multiple connected clients.

Q9. What is a WebSocket Group?

Answer: A group is a logical collection of WebSocket connections that can receive shared events.

Q10. What is Redis Channel Layer?

Answer: It is a Channels backend that uses Redis to distribute messages and events across application instances.

Q11. What is Django Channels?

Answer: Django Channels extends Django to support asynchronous protocols such as WebSocket.

Q12. What is a Consumer?

Answer: A Consumer is a Channels component that handles connection events and incoming messages.

Q13. What is the difference between SyncConsumer and AsyncConsumer?

Answer: SyncConsumer uses synchronous methods, while AsyncConsumer uses asynchronous methods and supports async I/O.

Q14. What is Ping and Pong?

Answer: Ping and Pong are WebSocket control frames used to check connection health.

Advanced Level

Q15. How do you scale WebSocket applications?

Answer: Use multiple application servers, a load balancer, a shared message layer such as Redis, and proper connection management.

Q16. How do you secure WebSocket?

Answer: Use WSS, authentication, authorization, origin validation, input validation, rate limiting, and message size limits.

Q17. What is Exponential Backoff?

Answer: It is a reconnection strategy where the waiting time increases after each failed attempt.

Q18. What is the difference between SSE and WebSocket?

Answer: SSE provides one-way server-to-client event streaming, while WebSocket provides bidirectional communication.

Q19. Does WebSocket use TCP?

Answer: The traditional WebSocket protocol uses TCP as its transport.

Q20. Can WebSocket send binary data?

Answer: Yes. WebSocket supports both text and binary messages.

Q21. Is WebSocket stateless?

Answer: The protocol connection is stateful because it remains open and maintains connection state.

Q22. Does WebSocket replace REST API?

Answer: No. REST is useful for request-response operations, while WebSocket is useful for continuous real-time communication. They can work together.

---

54. WebSocket Project Ideas

Beginner Projects

1. Echo Server

Features: - Client message পাঠাবে। - Server একই message ফিরিয়ে দেবে।

2. Simple Chat Application

Features: - Send message - Receive message - Real-time communication

Intermediate Projects

3. Group Chat Application

Features: - Multiple rooms - User join/leave - Group messaging - Message history

4. Live Notification System

Features: - Real-time notification - User-specific notification - Unread notification count

5. Live Order Tracking

Features: - Order status update - Delivery status - Real-time location update

Advanced Projects

6. Real-time Collaboration Tool

Features: - Multiple users - Live editing - User presence - Shared document state

7. Live Trading Dashboard

Features: - Live market updates - Real-time charts - Price notifications

8. Scalable Chat System

Features: - Django Channels - Redis - PostgreSQL - Authentication - Multiple chat rooms - Private messaging - Message history - Read receipts - Online/offline status

---

55. WebSocket Learning Roadmap

Level 1: Fundamentals

- HTTP basics
- Client-Server architecture
- Request-Response
- WebSocket protocol
- Handshake
- ws:// and wss://

Level 2: JavaScript WebSocket

- WebSocket constructor
- onopen
- onmessage
- onerror
- onclose
- send()

- close()

Level 3: Python WebSocket

- websockets package
- Async programming
- Client
- Server
- Send and receive

Level 4: FastAPI WebSocket

- WebSocket endpoint
- accept()
- receive_text()
- send_text()
- WebSocketDisconnect
- Connection manager

Level 5: Django Channels

- ASGI
- Consumer
- Routing
- Authentication
- Groups
- Broadcasting

Level 6: Redis

- Redis fundamentals
- Channel layer
- Group messaging
- Multiple server communication

Level 7: Advanced Concepts

- Security
- Reconnection
- Heartbeat
- Rate limiting
- Scaling
- Load balancing
- Monitoring

Level 8: Build Projects

- Chat application
- Notification system
- Live dashboard
- Scalable messaging system

---

56. Important Commands

Install FastAPI

```bash
pip install fastapi uvicorn
```

Run FastAPI

```bash
uvicorn main:app --reload
```

Install Django Channels

```bash
pip install channels
```

Install Redis Channel Layer

```bash
pip install channels-redis
```

Install Daphne

```bash
pip install daphne
```

Run Django ASGI Application

``` bash
daphne -p 8000 config.asgi:application
```

Install Python WebSockets

``` bash
pip install websockets
```

---

57. Final Summary

Topic	Key Point
WebSocket	Real-time communication protocol
Full-Duplex	Both sides communicate
Persistent Connection	Connection remains open
Handshake	Initial connection process
HTTP 101	Protocol switching accepted
ws://	Unencrypted WebSocket
wss://	Secure WebSocket
Ping/Pong	Connection health checking
Broadcasting	Send to multiple clients
Group	Collection of connections
FastAPI	Python WebSocket support
Django Channels	Django real-time support
Consumer	Handles WebSocket events
Redis	Cross-server message distribution
ASGI	Asynchronous Server Gateway Interface
SSE	One-way server-to-client events
Scaling	Multiple servers and shared messaging

Most Important Things to Remember

1. WebSocket provides bidirectional communication.
2. WebSocket connection is persistent.
3. WebSocket starts with a handshake.
4. HTTP 101 means protocol switching accepted.
5. `ws://` is unencrypted.
6. `wss://` is encrypted.
7. FastAPI supports WebSocket directly.
8. Django uses Channels for WebSocket support.

9. Redis helps distribute messages between servers.
10. Authentication and authorization are essential.
11. Production applications should use WSS.
12. WebSocket does not replace REST API.

Recommended Documentation

1. MDN WebSocket API: https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
2. FastAPI WebSocket: https://fastapi.tiangolo.com/advanced/websockets/
3. Django Channels: https://channels.readthedocs.io/
4. Python WebSockets: https://websockets.readthedocs.io/
5. WebSocket Protocol (RFC 6455): https://www.rfc-editor.org/rfc/rfc6455
6. Redis: https://redis.io/docs/latest/

End of WebSocket Complete Notes

Practice Goal: Build a real-time Chat Application using Django Channels + Redis + PostgreSQL.

Django Channels: Beginner to Advanced Full Notes 1. What is Django Channels? HTTP vs WebSocket Client request পাঠায়। Server response দেয়। সাধারণত প্রতিটি request আলাদা। Client ও server-এর মধ্যে persistent connection থাকে। দুই দি ক থেকে ই message পাঠানো যায়। Real-time application তৈরি তে ব্যবহার হয়। Django Channels কোথায় ব্যবহার হয়? Real-time chat application Live notifications Live dashboard Typing indicators Online user presence Live tracking Multiplayer games Real-time collaboration 2. Important Concepts ASGI WSGI vs ASGI WSGI: Traditional synchronous web applications। Django Channels হলো Django-এর একটি extension, যা WebSocket, real-time communication, background events এবং asynchronous communication support করে। সাধারণ Django HTTP request-response পদ্ধতিতে কাজ করে। Django Channels ব্যবহার করে client ও server-এর মধ্যে persistent connection তৈরি করা যায়। HTTP: WebSocket: ASGI-এর পূর্ণরূপ Asynchronous Server Gateway Interface। এটি asynchronous Python application চালানোর standard interface। Django Channels সাধারণত asgi.py ব্যবহার করে। ASGI: Asynchronous communication এবং WebSocket support করে। Consumer Routing Channel Layer Redis Group 3. Installation Consumer হলো Django Channels-এর সেই component, যা WebSocket connection ও message handle করে। Routing নির্ধারণ করে কোন WebSocket URL কোন

consumer-এর কাছে যা বে। Channel layer বিভিন্ন consumer-এর মধ্যে message পাঠানো র ব্যবস্থা করে। Redis একটি in-memory data store। Production environment-এ Django Channels-এর channel layer হিসেবে Redis বহুলহু ব্যবহৃত হয়। Group-এর সাহায্যে একাধিক connected client-কে একই message পাঠানো যায়। প্রথমে virtual environment তৈরি করো। python -m venv venv Windows: venv`\Scripts`{=tex}`\activate`{=tex} Packages install করো: pip install django channels channels-redis Project তৈরি করো: django-admin startproject config . python manage.py startapp chat Project structure: project/ ├── manage.py ├── config/ │ ├── **init**.py │ ├── settings.py │ ├── urls.py 4. Configure settings.py │ ├── asgi.py │ └── routing.py └── chat/ ├── **init**.py ├── consumers.py ├── routing.py ├── views.py └── urls.py config/settings.py ফা ইলে: INSTALLED_APPS = \["daphne", "django.contrib.admin", "django.contrib.auth", "django.contrib.contenttypes", "django.contrib.sessions", "django.contrib.messages", "django.contrib.staticfiles", "channels", "chat",\] ASGI_APPLICATION = "config.asgi.application" যদি daphne ব্যবহার করতে চা ও, তা হলে install করো: pip install daphne Production বা multi-process application-এর জন্য Redis channel layer configure করো: CHANNEL_LAYERS = { "default": { "BACKEND": "channels_redis.core.RedisChannelLayer", "CONFIG": { "hosts": \[("127.0.0.1", 6379)\], }, }, } এই configuration-এর জন্য Redis server চালু থাকতে হবে। 5. Create Your First Consumer connect() --- client connect করলে চলে। accept() --- WebSocket connection গ্রহণ করে। receive() --- client message পাঠালে চলে। send() --- client-কে message পাঠায়। disconnect() --- connection বন্ধ হলে চলে। 6. WebSocket Routing chat/consumers.py: from channels.generic.websocket import AsyncWebsocketConsumer class EchoConsumer(AsyncWebsocketConsumer): async def connect(self): await self.accept() await self.send(text_data="WebSocket Connected!") async def disconnect(self, close_code): print("WebSocket disconnected:", close_code) async def receive(self, text_data=None, bytes_data=None): if text_data is not None: await self.send(text_data=f"You sent: {text_data}") এখা নে: chat/routing.py: from django.urls import path from .consumers import EchoConsumer websocket_urlpatterns = \[path("ws/echo/", EchoConsumer.as_asgi()),\] config/routing.py: from chat.routing import websocket_urlpatterns **all** = \["websocket_urlpatterns"\] 7. Create a Test HTML Page config/asgi.py: import os from django.core.asgi import get_asgi_application from channels.routing import ProtocolTypeRouter, URLRouter from channels.auth import AuthMiddlewareStack from django.urls import path os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings") django_asgi_app = get_asgi_application() from chat.routing import websocket_urlpatterns application = ProtocolTypeRouter({ "http": django_asgi_app, "websocket": AuthMiddlewareStack(URLRouter(websocket_urlpatterns)), })

get_asgi_application() আগে initialize করা হয়েছে, যা তে Django app setup সম্পন্ন হওয়ার পর consumer ও model imports সঠিকভাবে কাজ করতে পারে। chat/views.py: from django.shortcuts import render def home(request): return render(request, "chat/index.html") chat/urls.py: from django.urls import path from . import views urlpatterns = \[path("", views.home, name="home"),\] config/urls.py: from django.contrib import admin from django.urls import path, include urlpatterns = \[path("admin/", admin.site.urls), path("", include("chat.urls")),\] HTML file তৈরি করো: chat/templates/chat/index.html \<!DOCTYPE html\>
```{=html}
<html lang="en">
```
```{=html}
<head>
```
```{=html}
<meta charset="UTF-8">
```
```{=html}
<title>
```
Django Channels Echo
```{=html}
</title>
```
```{=html}
</head>
```
```{=html}
<body>
```
```{=html}
<h1>
```
WebSocket Echo Test
```{=html}
</h1>
```
`<input id="messageInput" type="text">`{=html}
`<button onclick="sendMessage()">`{=html}Send`</button>`{=html}
```{=html}
<ul id="messages">
```
```{=html}
```


```
```{=html}
<script>
 const scheme = location.protocol === "https:"? "wss": "ws";
 const socket = new WebSocket(
 `${scheme}://${location.host}/ws/echo/`
 );
 socket.onopen = function () {
 console.log("Connected");
 };
 socket.onmessage = function (event) {
 const li = document.createElement("li");
 li.textContent = event.data;
 document.getElementById("messages").appendChild(li);
 };
 socket.onclose = function () {
8. Sync vs Async Consumers
Synchronous Consumer
Asynchronous Consumer
 console.log("Disconnected");
 };
 socket.onerror = function (error) {
 console.error("WebSocket error:", error);
 };
 function sendMessage() {
 const input = document.getElementById("messageInput");
 if (socket.readyState === WebSocket.OPEN) {
 socket.send(input.value);
 input.value = "";
 }
 }
 </script>
```
```{=html}
</body>
```
```{=html}
</html>
```
Run: python manage.py migrate python manage.py runserver Browser-এ যা ও: http://127.0.0.1:8000/ from channels.generic.websocket import WebsocketConsumer class MyConsumer(WebsocketConsumer): def connect(self): self.accept() def receive(self, text_data=None,
```

bytes_data=None): self.send(text_data="Hello from sync consumer!") Sync consumer সাধারণ synchronous code-এর জন্য। Async consumer asynchronous operations-এর জন্য। Async consumer-এর ভিতরে synchronous database operations সরাসরি চালালে event loop block হতে পারে। Async consumer-এ asynchronous ORM methods অথবা database_sync_to_async ব্যবহার করো। 9. JSON Messages from channels.generic.websocket import AsyncWebsocketConsumer class MyAsyncConsumer(AsyncWebsocketConsumer): async def connect(self): await self.accept() async def receive(self, text_data=None, bytes_data=None): await self.send(text_data="Hello from async consumer!") পার্থক্য: Client ও server-এর মধ্যে structured data পাঠানো র জন্য JSON ব্যবহার করা যায়। import json from channels.generic.websocket import AsyncWebsocketConsumer class JsonConsumer(AsyncWebsocketConsumer): async def connect(self): await self.accept() async def receive(self, text_data=None, bytes_data=None): try: data = json.loads(text_data or "{}") except json.JSONDecodeError: await self.send(text_data=json.dumps({ "error": "Invalid JSON" })) return message = data.get("message", "") 10. Channel Layer and Redis 11. Complete Group Chat Consumer await self.send(text_data=json.dumps({"message": message, "status": "success" })) Client থেকে: socket.send(JSON.stringify({ message: "Hello Django" })); একাধিক client-এর মধ্যে message broadcast করতে channel layer ব্যবহার করা হয়। In-memory channel layer development ও testing-এ কাজে লাগে, কিন্তু এটি production-এর multiprocess setup-এর জন্য উপযুক্ত নয়। Redis install: pip install channels-redis Redis server চালু করো এবং settings.py -এ configuration যোগ করো: CHANNEL_LAYERS = { "default": { "BACKEND": "channels_redis.core.RedisChannelLayer", "CONFIG": { "hosts": \[("127.0.0.1", 6379)\], }, }, } Redis channel layer একাধিক application process-এর মধ্যে event পাঠাতে সাহায্য করে। chat/consumers.py: import json from channels.generic.websocket import AsyncWebsocketConsumer class ChatConsumer(AsyncWebsocketConsumer): async def connect(self): Room Routing self.room_name = self.scope\["url_route"\]\["kwargs"\]\["room_name"\] self.room_group_name = f"chat_{self.room_name}" await self.channel_layer.group_add(self.room_group_name, self.channel_name) await self.accept() async def disconnect(self, close_code): await self.channel_layer.group_discard(self.room_group_name, self.channel_name) async def receive(self, text_data=None, bytes_data=None): try: data = json.loads(text_data or "{}") except json.JSONDecodeError: return message = data.get("message", "") if not isinstance(message, str) or not message.strip(): return await self.channel_layer.group_send(self.room_group_name, {"type": "chat_message", "message": message.strip(), }) async def chat_message(self, event): await self.send(text_data=json.dumps({ "message": event\["message"\] })) chat/routing.py: Group Methods

group_add() --- consumer-কে group-এ যুক্ত করে। group_discard() --- consumer-কে group থেকে সরায়। group_send() --- group-এর সবা ইকে event পাঠায়। chat_message() --- type="chat_message" event handle করে। 12. WebSocket Client for Chat from django.urls import path from .consumers import ChatConsumer websocket_urlpatterns = \[path("ws/chat/`<str:room_name>`{=html}/", ChatConsumer.as_asgi(),),\] গুরুত্বপূর্ণ: এই example-টি basic demonstration। এখা নে authentication, room permission, rate limiting এবং message persistence যোগ করা হয়নি। HTML-এর JavaScript অংশ: `<input id="messageInput" type="text">`{=html} `<button onclick="sendMessage()">`{=html}Send`</button>`{=html}

```{=html}
<ul id="messages">
```

```{=html}
</ul>
```

```{=html}
<script>
 const scheme = location.protocol === "https:"? "wss": "ws";
 const roomName = "general";
 const socket = new WebSocket(
 `${scheme}://${location.host}/ws/chat/${roomName}/`
 );
 socket.onmessage = function (event) {
 const data = JSON.parse(event.data);
 const li = document.createElement("li");
 li.textContent = data.message;
 document.getElementById("messages").appendChild(li);
13. Scope and Authentication
Authentication: user কে?
Authorization: user এই room-এ প্রবেশ করতে পা রবে কি?
 };
 function sendMessage() {
 const input = document.getElementById("messageInput");
 if (socket.readyState === WebSocket.OPEN) {
 socket.send(JSON.stringify({
 message: input.value
 }));
 input.value = "";
 }
 }
</script>
```

একই room name দি য়ে একাধিক browser tab খুলে test করো। একটি tab থেকে

message পাঠালে অন্য tabগুলো ও message পাবে। self.scope -এ connection সম্পর্কি ত তথ্য থাকে। উদাহরণ: user = self.scope\["user"\] room_name = self.scope\["url_route"\]\["kwargs"\]\["room_name"\] AuthMiddlewareStack Django session-based authentication-এমাধ্যমে WebSocket connection-এ user information উপলব্ধ করতে পারে। কিন্তু authentication এবং authorization এক জি নি স নয়: Protected chat room-এর ক্ষেত্রে connect() -এ permission যাচাই করো: from channels.generic.websocket import AsyncWebsocketConsumer class PrivateChatConsumer(AsyncWebsocketConsumer): async def connect(self): user = self.scope\["user"\] if not user.is_authenticated: 14. Database Operations in Async Consumers await self.close() return \# এখা নে room membership ও permission যাচাই করতে হবে। await self.accept() এটি শুধু authentication check-এর উদাহরণ। বাস্তব private room-এর জন্য membership authorizationও implement করতে হবে। Async consumer-এ synchronous ORM call সরাসরি ব্যবহার করা এড়িয়ে চলো। chat/models.py: from django.conf import settings from django.db import models class Room(models.Model): name = models.CharField(max_length=100, unique=True) def **str**(self): return self.name class Message(models.Model): room = models.ForeignKey(Room, on_delete=models.CASCADE, related_name="messages",) user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,) content = models.TextField() created_at = models.DateTimeField(auto_now_add=True) def **str**(self): return f"{self.user} - {self.room}" Migration: Database Helper 15. Notifications python manage.py makemigrations python manage.py migrate from channels.db import database_sync_to_async from .models import Message, Room @database_sync_to_async def save_message(room_name, user, content): room = Room.objects.get(name=room_name) return Message.objects.create(room=room, user=user, content=content,) @database_sync_to_async def get_recent_messages(room_name): room = Room.objects.get(name=room_name) return list(Message.objects.filter(room=room) .select_related("user") .order_by("-created_at")\[:50\]) Consumer থেকে ব্যবহার: message = await save_message(self.room_name, self.scope\["user"\], "Hello!",) Database access করার আগে user authenticated কি না এবং room-এ তার permission আছে কি না যাচাই করতে হবে। Room.DoesNotExist -এর মতো database errors-ও handle করো। Notification event পাঠানো র ধা রণা: 16. Typing Indicator await self.channel_layer.group_send(f"user_{user_id}", {"type": "notification_message", "message": "You have a new notification", }) Consumer event handler: async def notification_message(self, event): await self.send(text_data=json.dumps({ "type": "notification", "message": event\["message"\], })) Notification group-এ user-কে যুক্ত করার আগে authentication ও ownership যাচাই করতে হবে। অন্য user-এর notification group-এ unauthorized access দে ওয়া যা বে না। Client থেকে typing event: socket.send(JSON.stringify({ type: "typing", is_typing: true })); Server-এ event type অনুযায়ী আলাদা handling যোগ করো: if data.get("type")

== "typing": await self.channel_layer.group_send(self.room_group_name, { "type": "typing_event", "is_typing": bool(data.get("is_typing")), }) return Event handler: async def typing_event(self, event): await self.send(text_data=json.dumps({ 17. Online Presence 1. User connect করলে presence record update করা হয়। 2. Disconnect হলে record update করা হয়। 3. Multiple browser tabs বা devices থা কলে connection count বি বে চনা করা হয়। 4. Unexpected disconnection-এর জন্য expiry বা heartbeat ব্যবস্থা রাখা হয়। 18. WebSocket Security WebSocket URL-এ wss:// ব্যবহার করো। AllowedHostsOriginValidator বা উপযুক্ত OriginValidator ব্যবহার করো। Authentication যাচাই করো। Room membership ও permissions যাচাই করো। Message size limit রাখো। Rate limiting implement করো। Client input validate করো। Sensitive data log করা এড়িয়ে চলো। User-supplied content HTML হিসেবে render কো রো না। Redis-কে public internet-এ উন্মুক্ত রে খো না। Origin Validation Example "type": "typing", "is_typing": event\["is_typing"\], })) বাস্তব application-এ typing event-এর সঙ্গে sender identity যোগ করো এবং event spam সীমিত করো। Online presence তৈরি করতে সাধারণত: শুধু connect() এবং disconnect() দি য়ে online status নির্ভুল রাখা যায় না, কারণ network failure হলে disconnect event সব সময় প্রত্যাশিতভাবে নাও আসতে পারে। Production application-এ: config/asgi.py: from channels.security.websocket import AllowedHostsOriginValidator from channels.auth import AuthMiddlewareStack from channels.routing import ProtocolTypeRouter, URLRouter from django.core.asgi import get_asgi_application import os os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings") 19. Testing WebSocket Consumers django_asgi_app = get_asgi_application() from chat.routing import websocket_urlpatterns application = ProtocolTypeRouter({ "http": django_asgi_app, "websocket": AllowedHostsOriginValidator(AuthMiddlewareStack(URLRouter(websocket_urlpatterns))), }) Production-এ ALLOWED_HOSTS -এ সঠিক domain configure করো। Origin validation authorization-এর বিকল্প নয়। Install: pip install channels chat/tests.py: from channels.testing import WebsocketCommunicator from django.test import TransactionTestCase from config.asgi import application class EchoConsumerTests(TransactionTestCase): async def test_echo_connection(self): communicator = WebsocketCommunicator(application, "/ws/echo/",) connected, _ = await communicator.connect() self.assertTrue(connected) response = await communicator.receive_from() self.assertEqual(response, "WebSocket Connected!") 20. Common Errors Error: No route found WebSocket URL ভুল। websocket_urlpatterns configure করা হয়নি। URLRouter সঠিকভাবে setup করা হয়নি। Error: Cannot connect to Redis Redis server বন্ধ। Host বা port ভুল। Redis connection configuration ভুল। Error: SynchronousOnlyOperation Async context-এ synchronous ORM ব্যবহার করা হয়েছে। Async ORM methods ব্যবহার করো। অথবা database_sync_to_async ব্যবহার করো। Error: WebSocket closes immediately

Consumer-এ exception। Authentication বা Origin validation failure। Routing ভুল। Connection accept করা হয়নি। await communicator.send_to("Hello") response = await communicator.receive_from() self.assertEqual(response, "You sent: Hello") await communicator.disconnect() Run: python manage.py test Project-এর routing অনুযায়ী test URL পরিবর্তন করো। সম্ভাব্য কারণ: সম্ভাব্য কারণ: সম্ভাব্য কারণ: সমাধান: সম্ভাব্য কারণ: Error: group_send কাজ করছে না Redis/channel layer configuration ভুল। Consumer group-এ যোগ হয়নি। Event-এর type ও handler name মে লে নি। 21. Production Deployment 1. DEBUG = False রাখো। 2. ALLOWED_HOSTS configure করো। 3. Environment variables-এ secrets রাখো। 4. Redis channel layer configure করো। 5. ASGI-capable server ব্যবহার করো। 6. HTTPS/WSS এবং reverse proxy configure করো। 7. Logging ও monitoring setup করো। 8. Database connection এবং performance test করো। 9. WebSocket authentication ও authorization test করো। 10. Redis-এর security ও availability নিশ্চিত করো। 22. Interview Questions সম্ভাব্য কারণ: Production deployment-এর আগে: Development server: python manage.py runserver এটি development-এর জন্য। Production deployment-এ উপযুক্ত ASGI server ও infrastructure configure করতে হবে। Q1. What is Django Channels? Django Channels Django-তে WebSocket এবং asynchronous protocols support করার extension। Q2. What is ASGI? ASGI হলো asynchronous Python web application-এর standard interface। Q3. What is a Consumer? Consumer WebSocket connection, incoming message এবং outgoing event handle করে। Q4. What is a Channel Layer? Channel layer application-এর বিভিন্ন consumer-এর মধ্যে event/message আদান-প্রদান করতে সাহায্য 23. Practice Projects Echo WebSocket Live notification demo Group chat Typing indicator Online user presence Private messaging Persistent chat with database Role-based private rooms Live dashboard Reconnect handling করে। Q5. Why Redis? Redis multiple workers/process-এর মধ্যে channel layer messages আদান-প্রদানে ব্যবহৃত হয়। Q6. What is a Group? Group হলো একাধিক channel-এর collection, যেখানে broadcast event পাঠানো যায়। Q7. What is group_send()? এটি নির্দিষ্ট group-এর channel-গুলো তে event পাঠায়। Q8. What is scope? Scope connection-সম্পর্কি ত তথ্য ধা রণ করে, যেমন user ও URL parameters। Q9. What is the difference between authentication and authorization? Authentication user-এর পরি চয় যাচাই করে; authorization user-এর access permission যাচাই করে। Q10. How do you access the database in an async consumer? Async ORM methods অথবা database_sync_to_async ব্যবহার করে। Q11. Why use WebSockets? Persistent, bidirectional communication ও low-latency updates-এর জন্য। Q12. Is the in-memory channel layer suitable for production? সাধারণ multi-process production deployment-এর জন্য নয়; Redis-এর মতো shared channel layer ব্যবহার করা ভালো। Beginner: Intermediate: Advanced: Rate-limited real-time notification system 24. 12-Week Learning Roadmap 25. Official Documentation Django: https://docs.djangoproject.com/ Django Channels: https://channels.readthedocs.io/ Channels Tutorial:

https://channels.readthedocs.io/en/stable/tutorial/index.html Channels Deployment: https://channels.readthedocs.io/en/stable/deploying.html Redis: https://redis.io/docs/ Final Advice প্রতিটি project-এ authentication, authorization, input validation, testing ও error handling যোগ করার চে ষ্টা করো। Week 1--2: Python async/await, Django fundamentals, HTTP ও ASGI। Week 3--4: WebSocket basics, Consumers, Routing, Scope। Week 5--6: Channel Layer, Redis, Groups ও Chat Application। Week 7--8: Authentication, Authorization, Database integration। Week 9--10: Notifications, Typing Indicator, Presence ও Testing। Week 11: Security, Performance ও Error Handling। Week 12: Production deployment ও final project। প্রথমে EchoConsumer তৈরি করে WebSocket connection বুঝো। এরপর Redis ও Group Chat implement করো। তারপর authentication, database persistence, private rooms, automated tests এবং production deployment শেখো। এভাবে ধা পে ধা পে practice করলে Django Channels beginner থেকে advanced level পর্যন্ত শে খা সহজ হবে।